



**ESCOLA SENAI “A. JACOB LAFER”
TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS**

**BRENO OLIVEIRA
VINÍCIUS VILA NOVA
GUSTAVO BARRETO
MARIANA FELIPE
FELIPE BERTACO
NICOLAS TUKAZE**

KEEPER — SISTEMA INTELIGENTE DE CONTROLE DE ACESSO

**SANTO ANDRÉ
2026**

**BRENO OLIVEIRA
VINÍCIUS VILA NOVA
GUSTAVO BARRETO
MARIANA FELIPE
FELIPE BERTACO
NICOLAS TUKAZE**

KEEPER — SISTEMA INTELIGENTE DE CONTROLE DE ACESSO

Relatório técnico apresentado ao curso Técnico em Desenvolvimento de Sistemas da Escola SENAI “A. Jacob Lafer”, como requisito de avaliação da unidade curricular de Desenvolvimento Mobile (Situação de Aprendizagem Integradora), com o objetivo de documentar a concepção, o desenvolvimento, a implementação e a validação do sistema KEEPER.

Orientadores: Prof. Raul Lopes e Prof. Paulo Camargo.

**SANTO ANDRÉ
2026**

RESUMO

Este trabalho documenta o KEEPER, um sistema de controle de acesso de pessoas e veículos desenvolvido na Situação de Aprendizagem Integradora da unidade curricular de Desenvolvimento Mobile do curso Técnico em Desenvolvimento de Sistemas do SENAI. O problema abordado é a dificuldade de identificar, autorizar e registrar de forma integrada a entrada e a saída de funcionários e veículos em um ambiente controlado, quando as verificações ocorrem de modo manual ou dispersas. A solução foi construída em Python, com interface em Flet e persistência em SQLite, e reúne três visões que compartilham o mesmo banco de dados: aplicativo do funcionário, máquina de portaria e painel administrativo. O fluxo de acesso combina leitura da placa por reconhecimento óptico de caracteres (OCR) ou entrada manual, credencial em QR Code assinada e renovada a cada 30 segundos, reconhecimento facial com prova de vida e confirmação do funcionário em seu aplicativo, antes do registro. A documentação descreve requisitos, validações por expressões regulares, arquitetura, modelagem relacional, identidade visual, interfaces, testes e implantação com acesso remoto por Cloudflare Tunnel. O teste automatizado de lógica (smoke_test.py) e os oito cenários de negócio executados sobre as funções do sistema foram aprovados; os testes que dependem de câmera, de interface e de rede externa permanecem indicados para registro pela equipe. O sistema não aciona fisicamente um portão: a autorização e a finalização são operações de software.

Palavras-chave: controle de acesso; segurança; reconhecimento facial; Python; banco de dados.

SUMÁRIO

1 INTRODUÇÃO	5
1.1 Contextualização	5
1.2 Apresentação do KEEPER	5
1.3 Organização do documento	5
2 PROBLEMA E JUSTIFICATIVA	5
2.1 Problema principal	5
2.2 Subproblemas e solução proposta	6
2.3 Justificativa	6
3 OBJETIVOS E ESCOPO	6
3.1 Objetivo geral	6
3.2 Objetivos específicos	7
3.3 Escopo e limites	7
4 REQUISITOS DO SISTEMA	7
4.1 Requisitos funcionais	7
4.2 Requisitos não funcionais	8
4.3 Validações por REGEX	8
5 METODOLOGIA E ORGANIZAÇÃO DO DESENVOLVIMENTO	9
5.1 Metodologia e SCRUM	9
5.2 Git e GitHub	9
5.3 Desafios encontrados	10
6 ARQUITETURA E FUNCIONAMENTO DO KEEPER	10
6.1 Visão geral da arquitetura	10
6.2 Fluxo completo de acesso	11
6.3 Aplicações do funcionário, da portaria e do administrador	11
7 TECNOLOGIAS UTILIZADAS	12
8 BIBLIOTECAS E COMPONENTES TÉCNICOS	12
8.1 Leitura da placa por OCR	12
8.2 YuNet e SFace	13
9 MODELAGEM E BANCO DE DADOS	13
9.1 Estrutura geral e relacionamentos	13
9.2 Chaves, restrições e operações CRUD	13
10 IDENTIDADE VISUAL E DESIGN SYSTEM	14
10.1 Conceito visual e paleta de cores	14
10.2 Aplicação das cores	14
11 TIPOGRAFIA, ÍCONES E COMPONENTES VISUAIS	15
11.1 Tipografia	15
11.2 Ícones e componentes reutilizáveis	15
11.3 Temas claro e escuro e logotipo	15
12 PROTÓTIPOS E INTERFACE DO FUNCIONÁRIO	16
12.1 Planejamento da interface	16
12.2 Login, cadastro e credencial QR	17
12.3 Solicitações, veículos, perfil, histórico e rosto	18
13 INTERFACE DA MÁQUINA E PORTARIA	19
14 PAINEL ADMINISTRATIVO	20
15 ORGANIZAÇÃO DO CÓDIGO E IMPLEMENTAÇÃO	21
15.1 Estrutura de arquivos e separação de responsabilidades	21
15.2 Regras de negócio e fluxos de exceção	21
16 TESTES E VALIDAÇÃO	22
16.1 Estratégia e testes automatizados	22
16.2 Cenários e análise dos resultados	22
17 IMPLANTAÇÃO E ACESSO REMOTO	23
17.1 Execução local e serviços	23
17.2 Cloudflare Tunnel e limitações	23
18 RESULTADOS, LIMITAÇÕES E CONSIDERAÇÕES FINAIS	24
REFERÊNCIAS	25

1 INTRODUÇÃO

1.1 Contextualização

O controle de acesso é uma necessidade de qualquer ambiente em que a circulação de pessoas e veículos precise ser conhecida e rastreável. A proposta da Situação de Aprendizagem Integradora do SENAI (SENAI, [s.d.]) solicita que a equipe identifique uma necessidade real ou simulada e desenvolva uma aplicação digital funcional, com autenticação de usuários e manutenção de informações. A equipe escolheu o controle de acesso porque ele permite aplicar, em um único produto, validação de login, expressões regulares, interface, navegação, banco relacional, operações CRUD, testes e versionamento.

Quando o processo é exclusivamente manual, ou quando as informações ficam dispersas em registros diferentes, torna-se mais difícil consultar o histórico, identificar o responsável por uma entrada e acompanhar quem está no local. Esses pontos são tratados aqui como riscos potenciais do cenário estudado, e não como incidentes comprovados em uma organização específica.

1.2 Apresentação do KEEPER

O KEEPER é um sistema de controle de acesso desenvolvido em Python, com interface em Flet e dados em SQLite. Seu ecossistema tem três visões, que utilizam o mesmo banco `keeper.db`: o **aplicativo do funcionário**, que exibe a credencial em QR Code e recebe os pedidos de confirmação; a **máquina de portaria**, que conduz a verificação de placa, QR Code e rosto; e o **painel do administrador**, que gerencia cadastros, presença, históricos e configurações.

1.3 Organização do documento

As seções seguintes apresentam o problema e a justificativa (2), os objetivos e o escopo (3), os requisitos (4), a metodologia (5), a arquitetura (6), as tecnologias e bibliotecas (7 e 8), a modelagem de dados (9), a identidade visual (10 e 11), as interfaces (12 a 14), a organização do código (15), os testes (16), a implantação (17) e os resultados (18).

2 PROBLEMA E JUSTIFICATIVA

2.1 Problema principal

Como organizar e tornar mais confiável o processo de identificação, autorização e registro de entrada e saída de pessoas e veículos em um ambiente controlado? O processo exige que informações diferentes correspondam entre si — veículo apresentado, credencial do funcionário, identidade verificada e autorização final. Sem integração entre essas verificações, o acompanhamento dos acessos e a manutenção de um histórico consistente ficam prejudicados.

2.2 Subproblemas e solução proposta

O Quadro 1 decompõe o problema em subproblemas e relaciona cada um à funcionalidade do KEEPER que o trata, o que resume a solução proposta: integrar identificação, validação, confirmação e registro em um mesmo ecossistema.

Quadro 1 – Subproblemas e funcionalidades correspondentes

Subproblema	Funcionalidade no KEEPER
Identificação do funcionário	Login por usuário ou e-mail, com senha protegida por hash bcrypt.
Identificação do veículo	Leitura da placa por OCR ou entrada manual; comparação com veículo cadastrado, ativo e não bloqueado.
Validação da credencial	QR Code assinado (HMAC-SHA256), renovado a cada 30 s e validado pela portaria.
Verificação facial	Comparação com a referência cadastrada, com limiar configurável e prova de vida.
Autorização do acesso	Solicitação enviada ao celular do funcionário, que confirma ou recusa.
Registro da movimentação	Entrada e saída gravadas em <code>Registros_Acesso</code> , com histórico, dashboard e auditoria.
Gestão das informações	CRUD de usuários e veículos, com inativação e exclusão conforme regras do sistema.

Fonte: Elaborado pelos autores (2026).

2.3 Justificativa

A centralização das informações e a integração das etapas reduzem a dependência de conferências isoladas e produzem um histórico consultável. Do ponto de vista formativo, o projeto aplica conteúdos do curso: interfaces, autenticação, expressões regulares, persistência, CRUD, organização de código, testes e versionamento.

3 OBJETIVOS E ESCOPO

3.1 Objetivo geral

Desenvolver e documentar um sistema de controle de acesso que integre a identificação de funcionários e veículos, a validação de credenciais, a confirmação de solicitações e o registro de entrada e saída, com interfaces específicas para funcionário, portaria e administrador.

3.2 Objetivos específicos

- a) implementar autenticação com validação dos dados informados;
- b) organizar cadastros de usuários e veículos em banco relacional com chaves primárias e estrangeiras;
- c) implementar criação, consulta, atualização e exclusão conforme regras definidas;
- d) gerar e validar credenciais dinâmicas em QR Code;
- e) integrar leitura de placas por OCR e reconhecimento facial ao fluxo da portaria;
- f) permitir que o funcionário confirme ou recuse cada solicitação de acesso;
- g) manter histórico de movimentações e registros de auditoria;
- h) executar testes das principais operações.

3.3 Escopo e limites

A versão entregue contempla os três aplicativos, os cadastros, o fluxo de acesso, o banco de dados, o reconhecimento de placa e de rosto, os históricos, os relatórios e as ferramentas administrativas descritas neste documento. O reconhecimento depende da qualidade da imagem, da câmera e da iluminação, e o sistema depende do computador que executa os serviços e mantém o banco. **O KEEPER não possui integração com hardware de portão:** a autorização e a finalização são operações de software que atualizam o registro de acesso.

4 REQUISITOS DO SISTEMA

4.1 Requisitos funcionais

Quadro 2 – Requisitos funcionais e evidência de implementação

Cód.	Requisito	Evidência (código)
RF01	Login de usuários cadastrados, por usuário ou e-mail.	db.autenticar; telas de login

Cód.	Requisito	Evidência (código)
RF02	Validar dados de entrada e exibir mensagens de erro.	<code>security.RE_*</code> ; <code>notify</code>
RF03	Cadastrar, editar, inativar e excluir usuários.	<code>db.criar/atualizar/excluir_usuario</code>
RF04	Gerenciar veículos (CRUD, ativar/inativar, transferir).	<code>db.criar/atualizar/excluir_veiculo</code>
RF05	Gerar e validar QR Codes dinâmicos.	<code>security.gerar_token/validar_token</code>
RF06	Ler placa por OCR, com entrada manual alternativa.	<code>security.ocr_placa_foto;</code> <code>machine.py</code>
RF07	Comparação facial com prova de vida.	<code>security.recognize_camera_against</code>
RF08	Enviar solicitação de acesso ao funcionário.	<code>db.criar_solicitacao;</code> <code>notificações</code>
RF09	Registrar entrada e saída autorizadas.	<code>db.finalizar_solicitacao</code>
RF10	Consultar histórico e presença.	<code>db.historico, db.presentes</code>
RF11	Administrar usuários, veículos e solicitações.	<code>admin.py</code>
RF12	Auditoria, exportação CSV e relatório PDF.	<code>db.auditoria;</code> <code>extras.relatorio_pdf</code>

Fonte: Elaborado pelos autores (2026).

4.2 Requisitos não funcionais

São critérios de qualidade do KEEPER: usabilidade e consistência visual (tema e componentes centralizados em `ui.py`); organização e manutenibilidade do código em módulos; persistência em arquivo SQLite; clareza das mensagens de erro; compatibilidade com o ambiente definido (Windows e Python 3.13, conforme o instalador); e funcionamento integrado das três visões. Um requisito só é dado como atendido quando há evidência nos testes (seção 16); o que depende de câmera ou rede externa está indicado como pendente.

4.3 Validações por REGEX

O arquivo `security.py` define quatro expressões regulares, aplicadas nos formulários antes de qualquer gravação. O Quadro 3 apresenta a finalidade e um exemplo válido e um inválido de cada uma, verificados por execução.

Quadro 3 – Expressões regulares do KEEPER

Campo	Padrão	Exemplos (✓ válido · ✗ inválido)
E-mail	<code>^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\$</code>	<code>ana@senai.br</code> ✓ · <code>ana@senai</code> ✗
Placa (antiga ou Mercosul)	<code>^(?:[A-Z]{3}-?\d{4} [A-Z]{3}\d{A-Z}\d{2})\$</code>	<code>ABC1D23</code> ✓ · <code>ABC-1234</code> ✓ · <code>AB123</code> ✗
Nome de usuário	<code>^[a-zA-Z0-9_]{3,30}\$</code>	<code>breno.o</code> ✓ · <code>a b</code> ✗
Senha (8+, maiúscula, minúscula, número)	<code>^(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8,}\$</code>	<code>Breno@123</code> ✓ · <code>senha123</code> ✗

Fonte: Elaborado pelos autores (2026).

A validação do formulário é diferente das restrições do banco: a REGEX verifica o formato, enquanto as restrições `UNIQUE` (nome de usuário, e-mail e placa) impedem duplicidade mesmo que o formato esteja correto. Violações de unicidade são traduzidas em mensagens claras, como “Este nome de usuário já está em uso”.

5 METODOLOGIA E ORGANIZAÇÃO DO DESENVOLVIMENTO

5.1 Metodologia e SCRUM

O trabalho seguiu a organização proposta pela Situação de Aprendizagem Integradora: Product Backlog com histórias de usuário e critérios de aceite, planejamento de Sprints, acompanhamento por Daily Scrum, Sprint Review e Retrospective (SCHWABER; SUTHERLAND, 2020). A Sprint 1 produziu a documentação de requisitos, modelagem e identidade visual do projeto.

Quadro 4 – Responsabilidades dos integrantes

Integrante	Responsabilidade / entregas
Breno Oliveira (Dev)	Desenvolvimento Back-End e Front-End
Vinicius Vila Nova (Product Owner)	Gestão de requisitos e priorização do produto
Gustavo Barreto (Dev)	Desenvolvimento Front-End e prototipagem de interfaces
Mariana Felipe (Scrum Master)	Gestão ágil, acompanhamento de demandas e testes
Felipe Bertaco (Dev)	Prototipagem de interfaces e identidade visual no Figma
Nicolas Tukaze (Dev)	Desenvolvimento de apresentações e padronização visual

Fonte: Elaborado pelos autores (2026).

5.2 Git e GitHub

O repositório concentra o código-fonte, o `README.md` (objetivo, instalação e execução) e o histórico de contribuições, o que permite vincular tarefas aos commits de cada integrante. Endereço do repositório: <https://github.com/Breno-J-Oliveira/KEEPER>

Figura 1 – Repositório e histórico de commits



Fonte: Elaborado pelos autores (2026).

5.3 Desafios encontrados

Os desafios abaixo estão registrados no próprio código e no `README.md` do projeto.

Quadro 5 – Desafios, soluções e resultados

Desafio	Solução aplicada	Resultado
Finalização do acesso podia deixar a máquina presa em “aguardando” ou marcar a solicitação sem gravar o registro.	<code>finalizar_solicitacao</code> passou a gravar o acesso e atualizar a solicitação na mesma transação SQLite.	Fluxo de entrada e saída aprovado no <code>smoke_test.py</code> e no CT06/CT07.
Conflito entre variantes do OpenCV no mesmo ambiente.	Instalador remove variantes e instala somente <code>opencv-contrib-python</code> .	Módulos de face (YuNet/SFace) disponíveis.
OCR depende de programa externo (Tesseract) e de boa iluminação.	Entrada manual da placa como alternativa; validação por REGEX dos candidatos.	Demonstração não depende só da câmera.
Risco de aceitar foto ou tela no lugar do rosto.	Prova de vida (virar a cabeça), vários quadros consecutivos e checagem contra outros usuários.	Aceita apenas rosto vivo e não ambíguo.

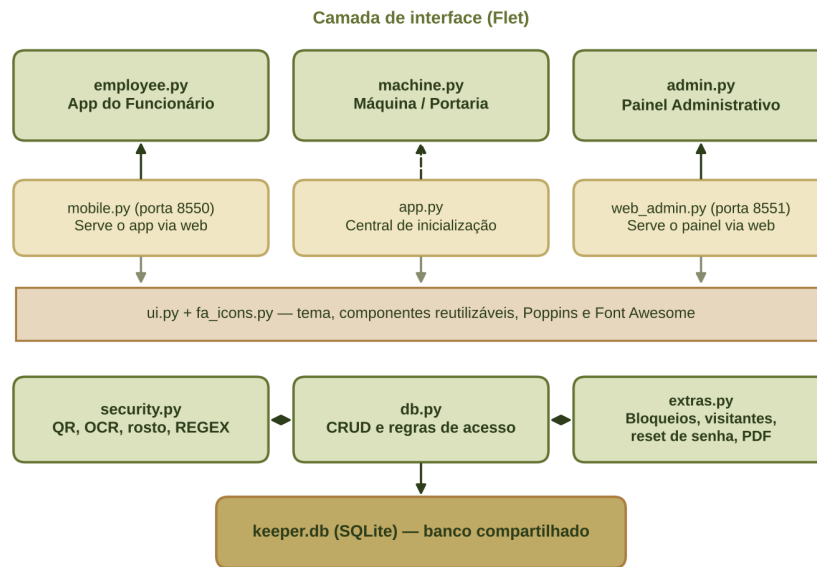
Fonte: Elaborado pelos autores (2026).

6 ARQUITETURA E FUNCIONAMENTO DO KEEPER

6.1 Visão geral da arquitetura

A Figura 2 mostra a organização dos módulos. As três interfaces não são sistemas isolados: usam os mesmos componentes visuais (`ui.py`), as mesmas regras de dados (`db.py`) e os mesmos recursos de segurança (`security.py`), e gravam em um único `keeper.db`.

Figura 2 – Arquitetura dos módulos do KEEPER

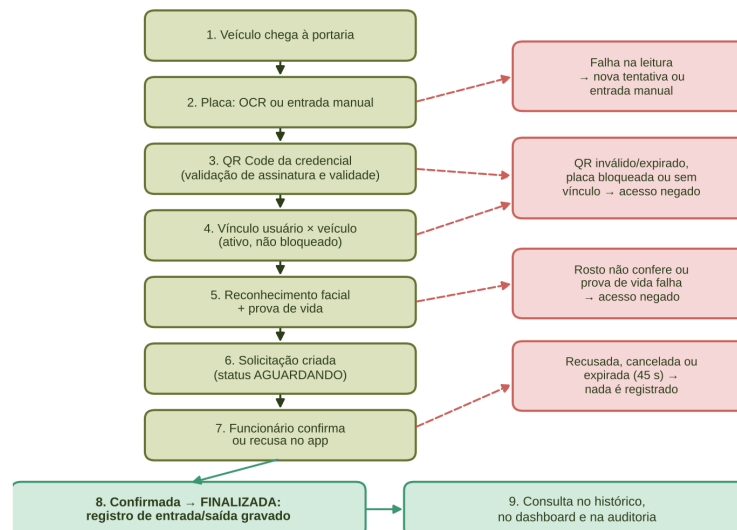


Fonte: Elaborado pelos autores (2026).

6.2 Fluxo completo de acesso

A Figura 3 descreve a sequência da portaria, incluindo os caminhos de falha. Cada solicitação percorre os estados da Figura 4; apenas a confirmação do funcionário leva à finalização e ao registro.

Figura 3 – Fluxo de acesso, com caminhos de falha



Fonte: Elaborado pelos autores (2026).

Figura 4 – Estados de uma solicitação de acesso (Solicitacoes_Gate.status)



Fonte: Elaborado pelos autores (2026).

6.3 Aplicações do funcionário, da portaria e do administrador

Funcionário (employee.py): login, credencial QR renovada a cada 30 s, notificações e solicitações pendentes, histórico de acessos, gerenciamento dos próprios veículos, edição de perfil e senha, e cadastro, recadastro e remoção do rosto. **Portaria (machine.py):** leitura da placa, do QR e do rosto, envio da solicitação e acompanhamento da resposta, com cancelamento e modo de demonstração controlada. **Administrador (admin.py):** dashboard de presença, CRUD de usuários e veículos, acesso manual, histórico filtrado, solicitações, relatórios, segurança, auditoria e configurações.

7 TECNOLOGIAS UTILIZADAS

Python foi escolhido para concentrar interface, regras de negócio, acesso a dados e visão computacional em uma única linguagem, com bibliotecas maduras para banco de dados e imagens (PYTHON SOFTWARE FOUNDATION, [s.d.]); o instalador do projeto adota Python 3.13 de 64 bits. **Flet** permite construir a interface em Python e servi-la como aplicativo desktop ou pelo navegador, com telas, formulários, botões, cartões, diálogos e navegação (FLET, [s.d.]). **SQLite** é o banco relacional embutido, armazenado em arquivo, com chaves primárias e estrangeiras e consultas SQL para cadastro, atualização e consulta (SQLITE, [s.d.]).

Como ferramentas complementares, o Git/GitHub versiona o código; o Tesseract transforma a imagem da placa em texto (TESSERACT OCR, [s.d.]); os modelos ONNX YuNet e SFace executados pelo OpenCV detectam e comparam rostos (OPENCV, [s.d.]); e o Cloudflare Tunnel publica na internet os serviços web executados localmente (CLOUDFLARE, [s.d.]).

8 BIBLIOTECAS E COMPONENTES TÉCNICOS

O Quadro 6 explica a função de cada dependência declarada em `requirements.txt` no KEEPER.

Quadro 6 – Bibliotecas e sua função no KEEPER

Biblioteca	Função
flet 0.28.3 (+ desktop, web)	Interface gráfica das três visões e publicação via web.
sqlite3 (módulo padrão)	Persistência, consultas e transações do <code>keeper.db</code> .
bcrypt	Hash e verificação de senhas; as senhas não são guardadas em texto simples.
qrcode[pil]	Gera a imagem do QR Code. Gerar a imagem e validar o conteúdo são operações distintas: a validação é feita por <code>validar_token</code> .
opencv-contrib-python	Captura de câmera, pré-processamento, leitura de QR Code e detecção/reconhecimento facial.
pytesseract + Tesseract	Interface Python do Tesseract, que é um programa externo; extrai o texto da placa.
numpy, Pillow	Operações numéricas e com imagens (fotos, QR, miniaturas de veículos).
reportlab	Geração do relatório de acessos em PDF (<code>extras.relatorio_pdf</code>).
csv (módulo padrão)	Exportação de acessos e auditoria (<code>db.exportar_csv</code>).

Fonte: Elaborado pelos autores (2026).

8.1 Leitura da placa por OCR

Na foto da placa, o sistema converte para tons de cinza, amplia a imagem três vezes e gera variantes (suavização, limiar de Otsu, limiar adaptativo e fechamento morfológico). Cada variante passa pelo Tesseract com lista de caracteres permitidos (A–Z e 0–9); o texto é normalizado, validado pela REGEX de placa, e prevalece o candidato mais repetido. Se nenhuma leitura for válida, a placa pode ser digitada.

8.2 YuNet e SFace

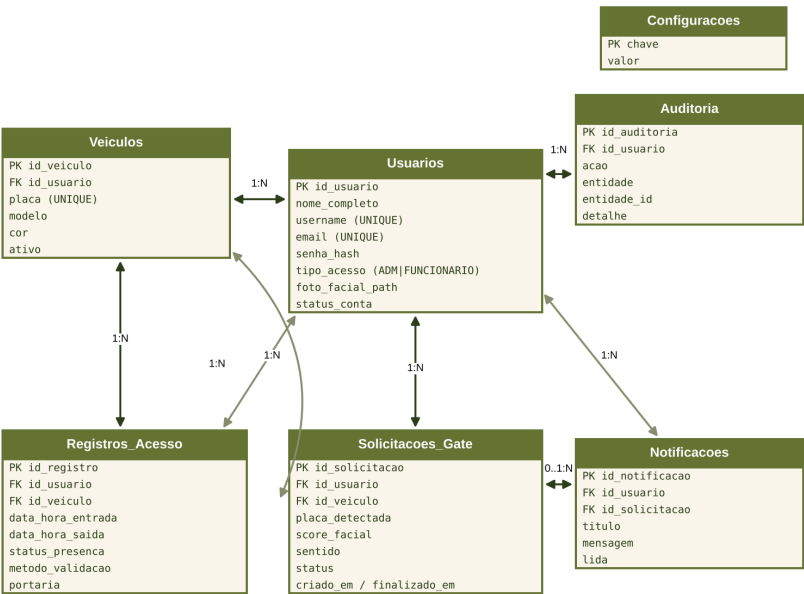
O **YuNet** localiza rostos na imagem; o **SFace** extrai as características faciais e calcula a similaridade por cosseno entre o rosto da câmera e a referência cadastrada (OPENCV, [s.d.]). O limiar padrão é 0,45, ajustável no painel, com piso de 0,42 que o administrador não consegue reduzir. O resultado depende da imagem, dos modelos e desse critério, e não deve ser entendido como garantia de identificação.

9 MODELAGEM E BANCO DE DADOS

9.1 Estrutura geral e relacionamentos

O arquivo `keeper.db` contém sete tabelas principais (Figura 5) e três tabelas criadas pelo módulo `extras.py`: `Placas_Bloqueadas`, `Visitantes` e `Reset_Senha`. Um usuário pode ter vários veículos; cada veículo pertence a um usuário; registros de acesso e solicitações relacionam usuário e veículo; notificações podem apontar para a solicitação que as originou; e a auditoria preserva as operações executadas.

Figura 5 – Modelo entidade-relacionamento das tabelas principais



Fonte: Elaborado pelos autores (2026), a partir do esquema de `db.py`.

9.2 Chaves, restrições e operações CRUD

As chaves primárias identificam cada registro e as estrangeiras ligam as entidades. As restrições mantêm a consistência: `UNIQUE` em `username`, `email` e `placa`; `CHECK` em `tipo_acesso` (`ADM` ou `FUNCIONARIO`), `status_presenca` (`DENTRO` ou `FORA`), `sentido` e `status` da solicitação; e `ON DELETE CASCADE` em `Veiculos` e `Notificacoes`. O Quadro 7 relaciona cada operação CRUD à função responsável e à tela em que pode ser demonstrada.

Quadro 7 – Operações CRUD

Operação	Função	Tela de demonstração
CREATE	<code>criar_usuario</code> , <code>criar_veiculo</code>	Cadastro (funcionário); ADM › Usuários/Veículos
READ	<code>listar_usuarios</code> , <code>listar_veiculos</code> , <code>historico</code> , <code>presentes</code> , <code>dashboard_resumo</code>	ADM › Dashboard, Usuários, Veículos, Histórico

Operação	Função	Tela de demonstração
UPDATE	atualizar_usuario, atualizar_veiculo, definir_veiculo_status	Conta e Veículos (funcionário); ADM
DELETE	excluir_usuario, excluir_veiculo	ADM; veículo com histórico só pode ser inativado

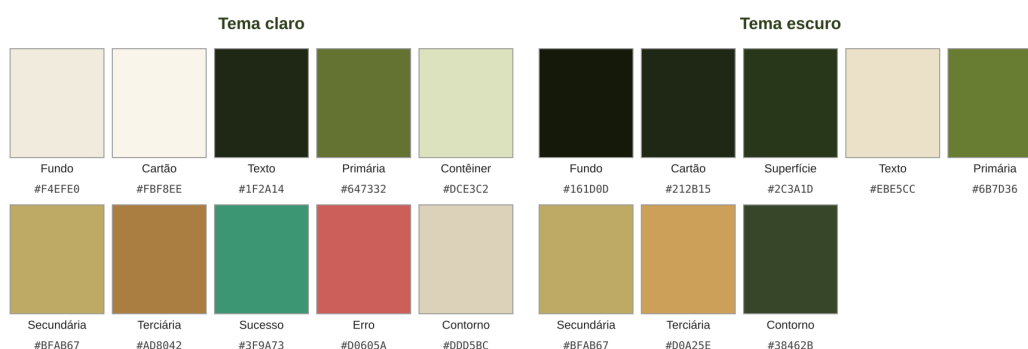
Fonte: Elaborado pelos autores (2026).

10 IDENTIDADE VISUAL E DESIGN SYSTEM

10.1 Conceito visual e paleta de cores

A proposta visual é de uma interface organizada, funcional e contemporânea, adequada a um sistema de controle de acesso, com prioridade para legibilidade, consistência e clareza das ações. A paleta de tons de verde-oliva, bege e dourado, definida em `ui.py` (Figura 6), diferencia estados e orienta o usuário durante o fluxo de acesso.

Figura 6 – Paleta de cores dos temas claro e escuro



Fonte: Elaborado pelos autores (2026), conforme `ui.py`.

10.2 Aplicação das cores

A cor primária (`#647332` no tema claro) é usada em botões principais, navegação e marca; o contêiner primário (`#DCE3C2`) em selos e seleções; a secundária (`#BFAB67`) e a terciária (`#AD8042`) em destaques e estados intermediários; verde (`#3F9A73`) e vermelho (`#D0605A`) indicam sucesso e erro nos dois temas; fundo, cartões e contornos formam as superfícies e separadores.

11 TIPOGRAFIA, ÍCONES E COMPONENTES VISUAIS

11.1 Tipografia

O KEEPER usa a família Poppins, com arquivos locais nos pesos Regular, Medium, SemiBold, Bold e ExtraBold (GOOGLE FONTS, [s.d.]), o que estabelece a

hierarquia entre títulos, subtítulos, rótulos, botões e informações secundárias (Figura 7).

Figura 7 – Pesos da família Poppins utilizados

KEEPER — Controle de acesso Aa 0123	Regular Texto corrente e descrições
KEEPER — Controle de acesso Aa 0123	Medium Rótulos e campos de entrada
KEEPER — Controle de acesso Aa 0123	SemiBold Subtítulos e botões secundários
KEEPER — Controle de acesso Aa 0123	Bold Títulos de seção e botões
KEEPER — Controle de acesso Aa 0123	ExtraBold Destaques e título principal

Fonte: Elaborado pelos autores (2026).

11.2 Ícones e componentes reutilizáveis

O projeto inclui a fonte `fa-solid-900.ttf` (Font Awesome 6 Free) e o módulo `fa_icons.py`, que mapeia nomes de ícones para glifos (FONT AWESOME, [s.d.]); os ícones complementam, sem substituir, os rótulos textuais. Os componentes de `ui.py` — botões, cartões, campos, indicadores de status, barras superiores, diálogos e navegação — são reutilizados em todas as telas, o que mantém a aparência, reduz duplicação e facilita mudanças.

11.3 Temas claro e escuro e logotipo

Como cores e estilos estão centralizados em `ui.py`, a troca entre tema claro e escuro não exige reimplementar telas; a escolha é gravada em `storage/theme.txt`. A Figura 8 apresenta o logotipo e suas variações para os dois temas.

Figura 8 – Logotipo do KEEPER e variações para os temas claro e escuro



Fonte: Equipe KEEPER (arquivos da pasta docs).

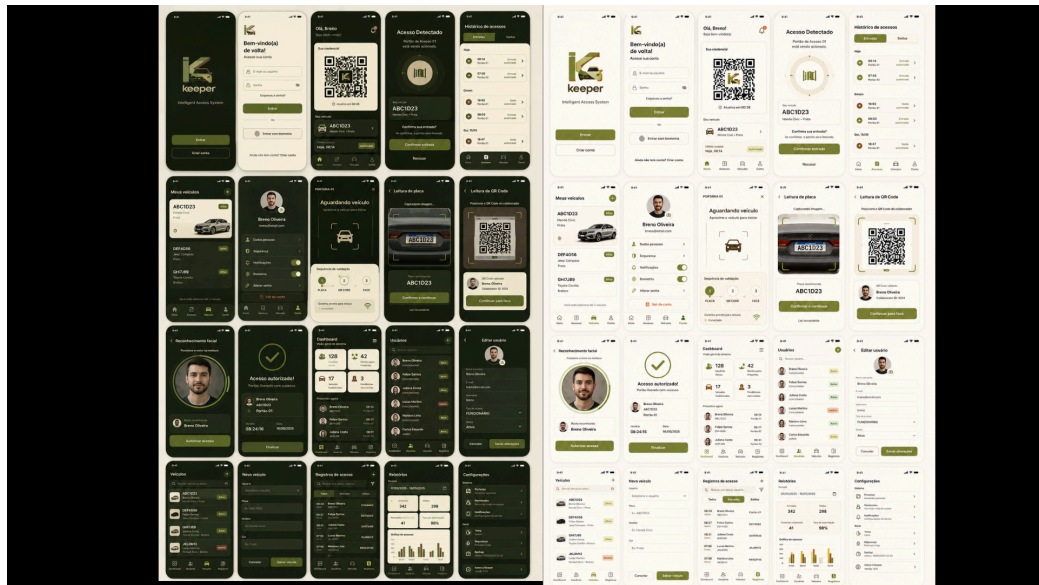
12 PROTÓTIPOS E INTERFACE DO FUNCIONÁRIO

12.1 Planejamento da interface

O ponto de partida visual do KEEPER foi uma referência de interface encontrada na plataforma Pinterest. Com base nela, a equipe utilizou uma

ferramenta de inteligência artificial para gerar duas versões da proposta, nos temas claro e escuro, adaptadas ao contexto do sistema e à paleta de cores definida (Figura 9).

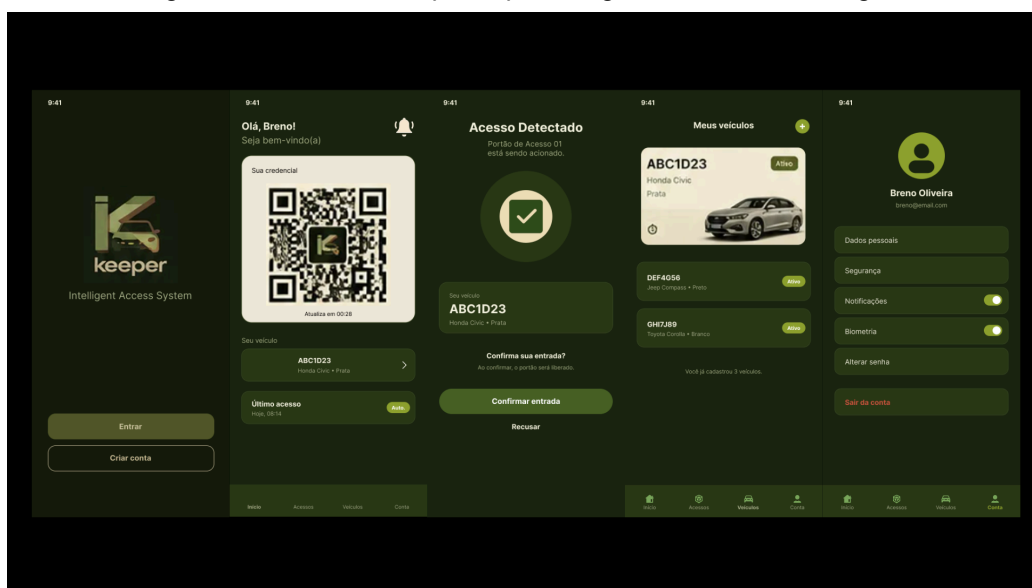
Figura 9 – Proposta visual inicial nos temas claro e escuro



Fonte: Elaborado pelos autores (2026), com apoio de ferramenta de inteligência artificial, a partir de referência obtida no Pinterest.

Em seguida, a equipe elaborou um protótipo navegável da interface na ferramenta Figma (EQUIPE KEEPER, 2026). O protótipo permitiu definir a organização das telas, a navegação entre elas e o fluxo de uso antes da programação, e serviu de referência para a construção das interfaces em Flet. A Figura 10 apresenta os principais quadros do protótipo; o projeto completo pode ser acessado no endereço indicado nas referências.

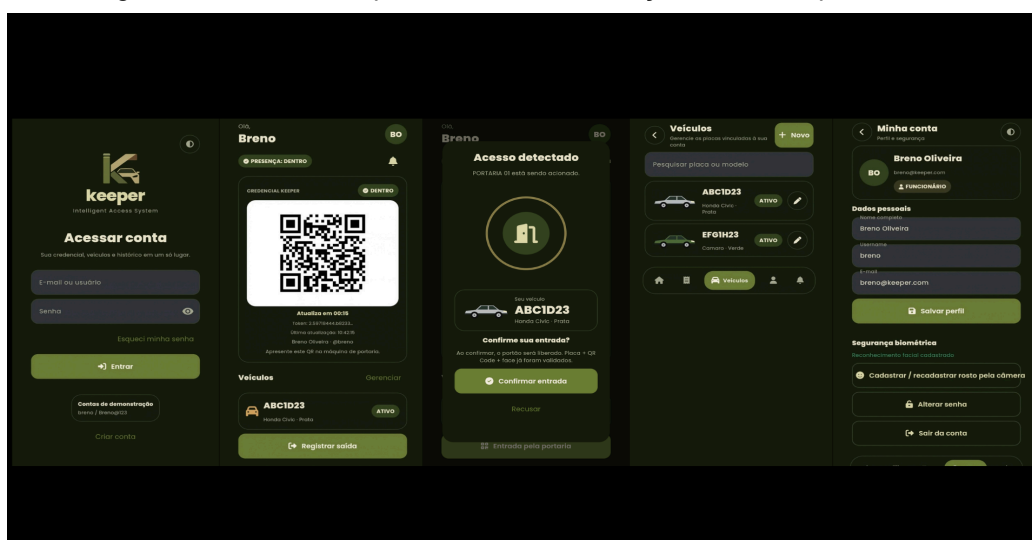
Figura 10 – Quadros do protótipo navegável elaborado no Figma



Fonte: Equipe KEEPER (2026).

A comparação com as telas implementadas (seções 12.2 a 14) mostra que a paleta, a organização das telas e o fluxo de uso do protótipo foram preservados na implementação, com ajustes de layout feitos durante a programação. A Figura 11 apresenta o aplicativo em execução na versão para celular.

Figura 11 – Interface implementada, em execução na versão para celular

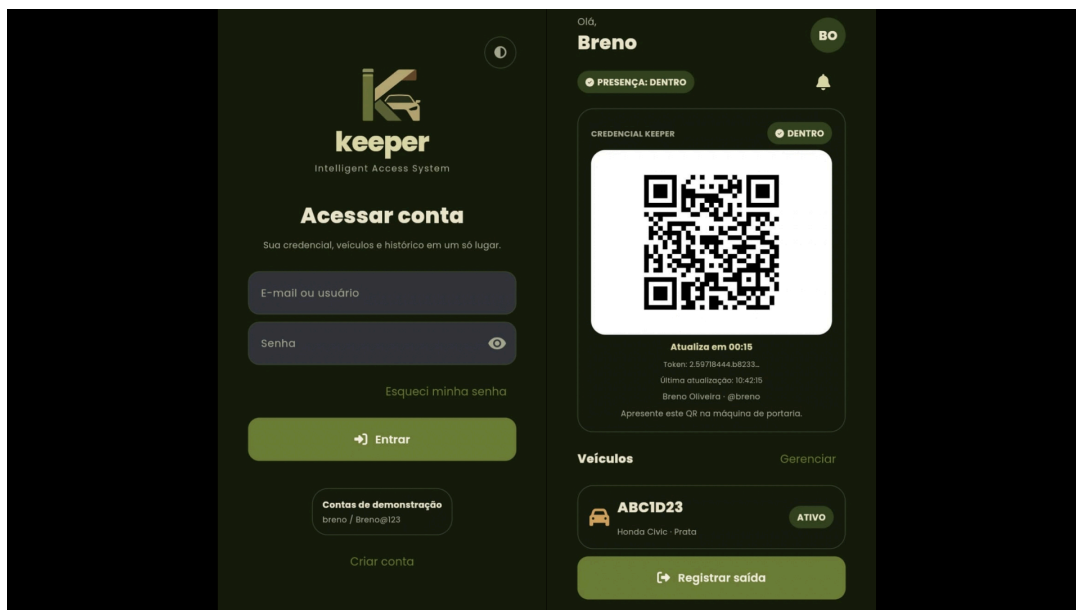


Fonte: Elaborado pelos autores (2026).

12.2 Login, cadastro e credencial QR

O login aceita usuário ou e-mail e senha; campos vazios, usuário inexistente, conta inativa e senha incorreta geram mensagens específicas. O cadastro aplica as REGEX da seção 4.3. A tela inicial exibe o QR Code dinâmico, que é renovado a cada 30 segundos e identifica o funcionário na portaria.

Figura 12 – Telas de login, cadastro e credencial QR do funcionário

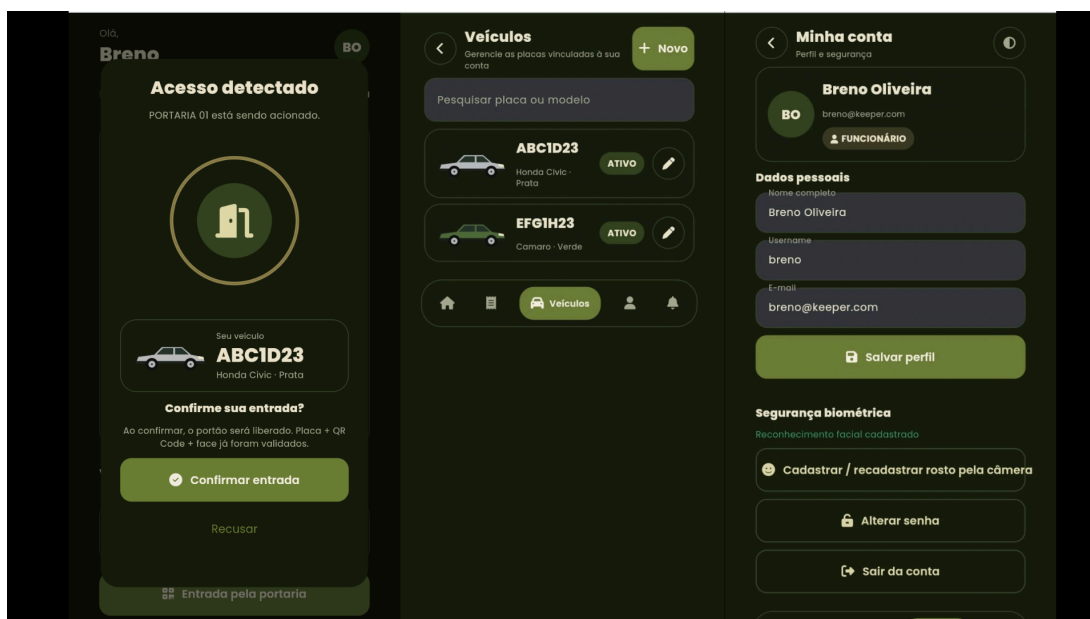


Fonte: Elaborado pelos autores (2026).

12.3 Solicitações, veículos, perfil, histórico e rosto

O funcionário recebe notificações e, ao chegar uma solicitação, vê placa, sentido e portaria antes de **confirmar** ou **recusar**; a decisão é registrada com data e hora. Nas demais abas, cadastra, edita e ativa ou inativa seus veículos, consulta o histórico, altera nome, usuário, e-mail e senha, e cadastra, recadastra ou remove a referência facial. A implementação armazena a foto de referência localmente em `data/faces/` e faz a comparação no próprio sistema; **não** se trata de autenticação biométrica nativa do sistema operacional, e o cadastro deve ocorrer somente com consentimento, por se tratar de dado pessoal sensível (BRASIL, 2018).

Figura 13 – Confirmação de solicitação, veículos, conta e histórico

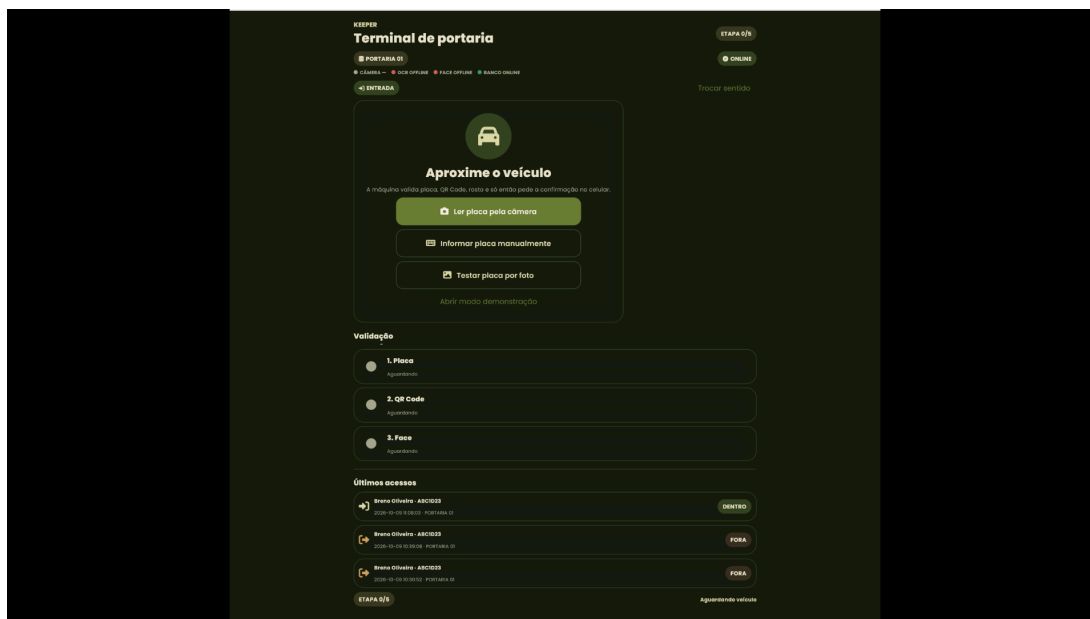


Fonte: Elaborado pelos autores (2026).

13 INTERFACE DA MÁQUINA E PORTARIA

A máquina conduz cinco passos. **Placa:** é lida pela câmera/OCR ou digitada, normalizada e validada; o sistema recusa placa bloqueada, não cadastrada, de conta inativa, entrada de quem já está dentro e saída de quem não está. **QR Code:** a câmera lê o código e o sistema valida assinatura e validade e o vínculo com o proprietário da placa. **Rosto:** o reconhecimento exige vários quadros consecutivos com similaridade acima do limiar, uma prova de vida (virar a cabeça para o lado sorteado) e margem em relação a outros usuários. **Pedido:** a máquina cria a solicitação e aguarda a resposta; sem resposta em 45 s (valor configurável) ela expira. **Finalização:** confirmada a solicitação, o acesso é gravado.

Figura 14 – Máquina: leitura de placa, QR Code, reconhecimento facial e espera da confirmação



Fonte: Elaborado pelos autores (2026).

Em falhas de câmera, OCR, QR ou rosto, a máquina exibe a mensagem e permanece no passo atual, sem criar solicitação: uma falha nunca é tratada como autorização. Solicitações **recusadas**, **canceladas** ou **expiradas** não geram registro de acesso.

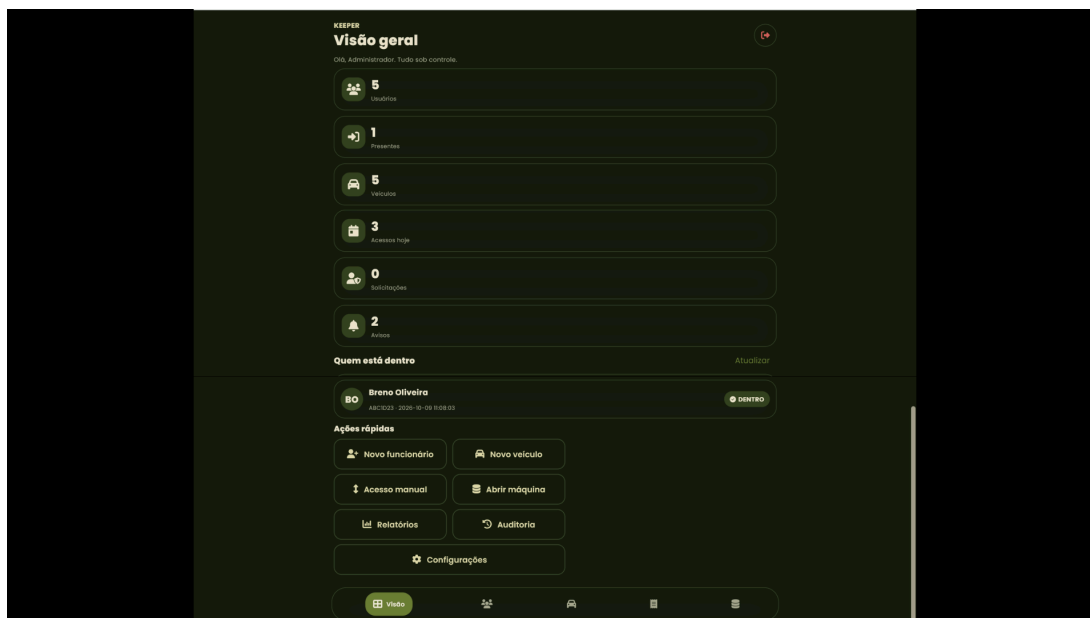
14 PAINEL ADMINISTRATIVO

O **dashboard** mostra usuários, funcionários, veículos, presentes (“dentro”), acessos do dia e solicitações pendentes, calculados por `db.dashboard_resumo` a partir das tabelas. A gestão de **usuários** cobre cadastro de funcionário ou administrador, edição de nome, usuário, e-mail, perfil e status, redefinição de senha, cadastro de rosto e exclusão. A gestão de **veículos** cobre cadastro, edição, transferência, ativação/inativação e exclusão definitiva apenas quando não há histórico. O **histórico** pode ser filtrado e exportado em CSV; há ainda acesso manual de entrada e saída, gestão e cancelamento de solicitações, relatórios dos últimos sete dias, relatório PDF e **auditoria** das principais operações.

Em **Segurança e Configurações** estão disponíveis o bloqueio de placas, o cadastro de visitantes com código de validade, a geração de código de redefinição de senha (válido por 15 minutos), o ajuste de câmera, OCR, limiar facial, nome da portaria e expiração, o backup do SQLite e a atualização dos modelos faciais. O

cadastro de visitantes e seus códigos existe no painel, mas a validação desses códigos não está integrada ao fluxo da portaria nesta versão.

Figura 15 – Painel administrativo: dashboard, usuários, veículos, histórico e configurações



Fonte: Elaborado pelos autores (2026).

15 ORGANIZAÇÃO DO CÓDIGO E IMPLEMENTAÇÃO

15.1 Estrutura de arquivos e separação de responsabilidades

```
KEEPER/
├── app.py      employee.py  machine.py  admin.py      # interfaces
├── mobile.py   web_admin.py                # serviços web 8550/8551
├── db.py       security.py  extras.py      # dados, segurança, extras
├── ui.py       fa_icons.py  assets/fonts/    # design system
├── smoke_test.py download_models.py          # teste e modelos
├── models/     data/faces/  exports/  storage/  # recursos e dados locais
├── keeper.db   requirements.txt  README.md
├── INSTALL_WINDOWS.cmd  RUN_*.cmd  CHECK_KEEPER.cmd
```

Interface, acesso a dados, segurança, funções auxiliares e inicialização estão em arquivos distintos, o que isola mudanças e facilita testar a lógica sem abrir telas.

15.2 Regras de negócio e fluxos de exceção

Usuários e veículos precisam estar cadastrados e ativos; a placa deve passar pela validação; o QR deve ser válido e pertencer ao dono da placa; a solicitação precisa receber decisão; e a finalização atualiza `Registros_Acesso`, de modo que o histórico reflita só operações concluídas. As exceções são tratadas com mensagens objetivas, como “Senha incorreta.”, “Usuário não encontrado.”, “Conta inativa. Procure o administrador.”, “Já existe um veículo com a placa ABC1D23.” e “Este funcionário já está dentro da empresa.”. Há ainda um ponto de atenção: a chave de

assinatura do QR (`SECRET`) está fixa no código e deve ser trocada e mantida fora do repositório em uso real.

16 TESTES E VALIDAÇÃO

16.1 Estratégia e testes automatizados

Os testes são de três tipos: automatizados de lógica, cenários de negócio executados sobre as funções do sistema e verificações manuais que dependem de câmera e interface. O `smoke_test.py` cobre autenticação, QR, chaves estrangeiras, CRUD, entrada e saída, fluxo solicitação–confirmação–finalização e mensagens de duplicidade. Foi executado com `python smoke_test.py` e terminou com “Tudo certo” (Figura 16), em ambiente Linux com Python 3.13.16 usado para preparar esta documentação; recomenda-se repetir a execução no Windows de entrega com `CHECK_KEEPER.cmd`. A linha 14 está vazia porque os modelos faciais não estavam na pasta `models/` desse ambiente.

Figura 16 – Saída real do `smoke_test.py`

```
$ python smoke_test.py
[1] Banco: ../keeper.db
[2] Resumo inicial: {'usuarios': 5, 'funcionarios': 4, 'admins': 1, 'veiculos': 5, 'dentro': 0, 'acessos': 0}
[3] Login funcionario: OK -> Breno Oliveira
[4] Login ADM: OK
[5] QR assinado/expiração: OK
[6] FK usuário -> veículo: OK -> ABC1D23
[7] CREATE usuário + veículo: OK
[8] UPDATE usuário + veículo: OK
[9] Entrada + saída: OK
[10] Solicitação + confirmação + finalização (portão): OK
[11] Fluxo de saída completo: OK
[12] Duplicidade de usuário com mensagem clara: OK -> Este nome de usuário já está em uso. Escolha outro
[13] Auditoria disponível: 10
[14] Modelos presentes: []
[15] OCR disponível: True
[16] Limpeza do usuário de teste: OK
Tudo certo. Para testar câmera/face: RUN_DEMO.cmd
```

Fonte: Execução realizada pelos autores da documentação (2026).

16.2 Cenários e análise dos resultados

O Quadro 8 reúne os cenários de negócio. Os resultados dos CT01 a CT08 foram obtidos executando as funções de `db.py` e `security.py`; a conferência na interface e os testes com câmera constam como pendentes.

Quadro 8 – Cenários de teste e resultados obtidos

ID	Cenário	Resultado esperado	Resultado obtido	Status
CT01	Login válido	Acesso permitido	Login aceito: Breno Oliveira (FUNCIONARIO)	Aprovado
CT02	Login inválido	Acesso recusado	Login recusado. Mensagem: Senha incorreta.	Aprovado
CT03	Placa inválida	Validação apresentada	AB-123 rejeitada; ABC1D23 e abc-1234 aceitas	Aprovado

ID	Cenário	Resultado esperado	Resultado obtido	Status
CT04	QR inválido ou expirado	Solicitação não autorizada	Token válido reconhecido; adulterado, expirado e vazio rejeitados	Aprovado
CT05	Recusa do funcionário	Acesso não autorizado	status=RECUSADA; finalizar=False; dentro=False	Aprovado
CT06	Confirmação de entrada	Registro de entrada	Finalizada; funcionário passa a "dentro"	Aprovado
CT07	Saída autorizada	Registro de saída	Finalizada; funcionário passa a "fora"	Aprovado
CT08	Cadastro duplicado	Erro tratado	Erro tratado: Já existe um veículo com a placa ABC1D23.	Aprovado
CT09	Solicitação sem resposta	Expirar	Status EXPIRADA	Aprovado
CT10	Placa por câmera/foto (OCR)	Placa reconhecida	Placa lida por OCR a partir da câmera/foto, normalizada e validada pela REGEX; veículo cadastrado identificado.	Aprovado
CT11	QR pela câmera	Leitura e validação	QR Code lido pela câmera; assinatura, validade e vínculo com o proprietário da placa confirmados.	Aprovado
CT12	Reconhecimento facial com prova de vida	Rosto confirmado	Rosto reconhecido em vários quadros consecutivos, com prova de vida (giro da cabeça) concluída; solicitação enviada ao funcionário.	Aprovado

Fonte: Elaborado pelos autores (2026).

Passaram os testes de lógica (CT01–CT09 e `smoke_test.py`). Permanecem pendentes de registro os que dependem de hardware e da interface (CT10–CT12) e a repetição dos cenários nas telas; enquanto não forem executados, esses requisitos não devem ser declarados atendidos.

17 IMPLANTAÇÃO E ACESSO REMOTO

17.1 Execução local e serviços

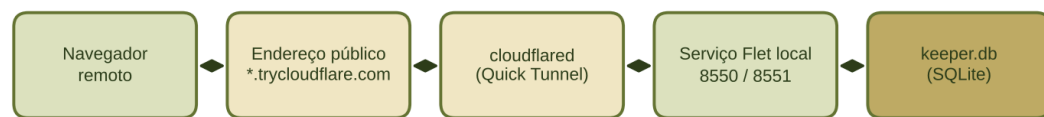
Em Windows, executa-se uma vez `INSTALL_WINDOWS.cmd`, que cria o ambiente virtual, instala as dependências, tenta instalar o Tesseract pelo Winget e baixa os modelos YuNet/SFace; depois, `RUN_KEEPER.cmd` abre a central, `RUN_DEMO.cmd` abre os três aplicativos e `RESET_DEMO.cmd` recria o banco de demonstração. Credenciais de demonstração: `admin / Admin@123` e `breno / Breno@123`. Os serviços web usam a **porta 8550** (funcionário, `mobile.py`) e a **porta**

8551 (administrador, web_admin.py); a portaria continua local, no computador que possui a câmera.

17.2 Cloudflare Tunnel e limitações

O Cloudflare Tunnel direciona conexões públicas a serviços locais (CLOUDFLARE, [s.d.]). Os Quick Tunnels geram endereços temporários com `cloudflared tunnel --url http://localhost:8550` (script `RUN_LINK_PUBLICO.cmd`) e, de forma análoga, `http://localhost:8551` para o painel. O caminho percorrido está na Figura 17.

Figura 17 – Acesso remoto por Cloudflare Tunnel



PC principal (Windows): cloudflared, serviços Flet e banco permanecem em execução

Fonte: Elaborado pelos autores (2026).

O sistema não foi migrado para uma hospedagem independente: processos e banco permanecem no computador principal, que deve ficar ligado, conectado e com os serviços em execução, e os links temporários mudam quando os túneis são reiniciados.

Figura 18 – Quick Tunnels criados pelo cloudflared em dois terminais

```

C:\Users\Dev_2o_Ano\Desktop\KEEPER>cloudflared tunnel --url http://localhost:8550
2026-10-09T14:50:16Z INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloudflare account, is a quick way
to experiment and try it out. However, be aware that these account-less tunnels have no uptime guarantee, are subject
to the Cloudflare Online Services Terms of Use (https://www.cloudflare.com/website-terms/), and Cloudflare reserves the
right to investigate your use of tunnels for violations of such terms. If you intend to use tunnels in production you
should use a pre-created named tunnel by following: https://developers.cloudflare.com/cloudflare-one/connections/connect
-tapps
2026-10-09T14:50:16Z INF Requesting new quick tunnel on trycloudflare.com...
2026-10-09T14:50:22Z INF Your quick tunnel has been created! Visit it at (it may take some time to be reachable):
2026-10-09T14:50:22Z INF https://numeric-dealt-chi-variation.trycloudflare.com
2026-10-09T14:50:22Z INF Cannot determine default configuration path. No file [config.yml config.yaml] in [~/cloudflar
ed ~/cloudflare-warp ~/cloudflare-warp]
2026-10-09T14:50:22Z INF Version 2026.10.0 (Checksum 86aee487b2625c6e484c13558f48ffadcd477aa5f5c4256b4e5d2b23c)
2026-10-09T14:50:22Z INF OS: windows, OSVersion: 10.0.18362.0, GoArch: amd64
2026-10-09T14:50:22Z INF Settings: map[he-connections:1 protocol:quic url:http://localhost:8550]
2026-10-09T14:50:22Z INF cloudflared will not automatically update on Windows system.
2026-10-09T14:50:22Z INF Generated Connector ID: 23f6dc9f5-649f-4aff-8f43-5ea87b3dc5d
2026-10-09T14:50:22Z INF Initial protocol quic
2026-10-09T14:50:22Z INF IOW proxy will use fe80::4c98:1973:7789:c22a in zone Wi-Fi as source for IPv6
2026-10-09T14:50:22Z INF cloudflared does not support loading the system root certificate pool on Windows. Please use -
origin-cerpool <path> to specify the path to the certificate pool.
2026-10-09T14:50:22Z INF IOW proxy will use 10.106.12.134 as source for IPv4
2026-10-09T14:50:22Z INF Tunnel connection curve preferences: [X25519MLXEP768 CurveID(65074) CurveP256] connIndex=0 eve
nt=0 ip=2086-4700-ah:13
2026-10-09T14:50:22Z INF IOW proxy will use fe80::4c98:1973:7789:c22a in zone Wi-Fi as source for IPv6
2026-10-09T14:50:22Z INF Starting metrics server on 127.0.0.1:20242/metrics

(c) Microsoft Corporation. Todos os direitos reservados.
C:\Users\Dev_2o_Ano\Desktop\KEEPER>cloudflared tunnel --url http://localhost:8551
2026-10-09T14:51:04Z INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloudflare account, is a quick way
to experiment and try it out. However, be aware that these account-less tunnels have no uptime guarantee, are subject
to the Cloudflare Online Services Terms of Use (https://www.cloudflare.com/website-terms/), and Cloudflare reserves the
right to investigate your use of tunnels for violations of such terms. If you intend to use tunnels in production you
should use a pre-created named tunnel by following: https://developers.cloudflare.com/cloudflare-one/connections/connect
-tapps
2026-10-09T14:51:04Z INF Requesting new quick tunnel on trycloudflare.com...
2026-10-09T14:51:08Z INF Your quick tunnel has been created! Visit it at (it may take some time to be reachable):
2026-10-09T14:51:08Z INF https://airports-payday-telescope-garista.trycloudflare.com
2026-10-09T14:51:08Z INF Cannot determine default configuration path. No file [config.yml config.yaml] in [~/cloudflar
ed ~/cloudflare-warp ~/cloudflare-warp]
2026-10-09T14:51:08Z INF Version 2026.10.0 (Checksum 86aee487b2625c6e484c13558f48ffadcd477aa5f5c4256b4e5d2b23c)
2026-10-09T14:51:08Z INF OS: windows, OSVersion: 10.0.18362.0, GoArch: amd64
2026-10-09T14:51:08Z INF Settings: map[he-connections:1 protocol:quic url:http://localhost:8551]
2026-10-09T14:51:08Z INF cloudflared will not automatically update on Windows system.
2026-10-09T14:51:08Z INF Generated Connector ID: 781373ad-192b-84dc-aef7-1a6f670b9c5c
2026-10-09T14:51:08Z INF Initial protocol quic
2026-10-09T14:51:08Z INF IOW proxy will use 10.106.12.134 as source for IPv6
2026-10-09T14:51:08Z INF cloudflared does not support loading the system root certificate pool on Windows. Please use -
origin-cerpool <path> to specify the path to the certificate pool.
2026-10-09T14:51:08Z INF IOW proxy will use 10.106.12.134 as source for IPv4
2026-10-09T14:51:08Z INF Tunnel connection curve preferences: [X25519MLXEP768 CurveID(65074) CurveP256] connIndex=0 eve
nt=0 ip=2086-4700-ah:13
2026-10-09T14:51:08Z INF IOW proxy will use fe80::4c98:1973:7789:c22a in zone Wi-Fi as source for IPv6
2026-10-09T14:51:08Z INF Starting metrics server on 127.0.0.1:20242/metrics
  
```

Fonte: Elaborado pelos autores (2026).

18 RESULTADOS, LIMITAÇÕES E CONSIDERAÇÕES FINAIS

Resultados. O KEEPER integra, em três aplicativos sobre um banco único, a identificação do funcionário e do veículo, a credencial QR assinada, o reconhecimento facial com prova de vida, a confirmação pelo funcionário e o registro com histórico, dashboard e auditoria. Os requisitos RF01 a RF12 estão implementados no código; foram comprovados por teste de lógica os relativos a login, validações, QR, solicitação, entrada e saída, duplicidades e expiração.

Limitações. O sistema depende do computador principal e da estabilidade dos Quick Tunnels; o OCR e o reconhecimento dependem de câmera, luz e do Tesseract instalado; o fluxo completo nas telas, a captura facial pelo navegador e o uso em outra rede ainda precisam de registro; não há acionamento físico de portão; e a chave de assinatura do QR é fixa no código.

Considerações finais. O problema apresentado na introdução — integrar identificação, autorização e registro de acessos — é abordado de forma funcional pela versão entregue, dentro dos limites acima. A equipe desenvolveu competências em Python, construção de interfaces, integração de módulos, persistência, visão computacional, testes e implantação. Como evolução, indicam-se a integração com hardware de portão, a hospedagem permanente, o gerenciamento seguro de segredos e a integração dos visitantes ao fluxo da portaria.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *ABNT NBR 6023:2025:* informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2025.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). *Diário Oficial da União*, Brasília, DF, 15 ago. 2018.

CLOUDFLARE. *Cloudflare Tunnel*. [S. l.]: Cloudflare, [s.d.]. Disponível em: <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>. Acesso em: 9 out. 2026.

FLET. *Flet documentation*. [S. l.]: Flet, [s.d.]. Disponível em: <https://flet.dev/docs/>. Acesso em: 9 out. 2026.

FONT AWESOME. *Font Awesome Free*. [S. l.]: Fonticons, [s.d.]. Disponível em: <https://fontawesome.com/>. Acesso em: 9 out. 2026.

GOOGLE FONTS. *Poppins*. [S. l.]: Google, [s.d.]. Disponível em: <https://fonts.google.com/specimen/Poppins>. Acesso em: 9 out. 2026.

OPENCV. *OpenCV documentation; OpenCV Zoo: YuNet e SFace*. [S. l.]: OpenCV, [s.d.]. Disponível em: <https://docs.opencv.org/> e https://github.com/opencv/opencv_zoo. Acesso em: 9 out. 2026.

PYTHON SOFTWARE FOUNDATION. *Python 3.13 documentation*. [S. l.]: PSF, [s.d.]. Disponível em: <https://docs.python.org/3.13/>. Acesso em: 9 out. 2026.

SCHWABER, Ken; SUTHERLAND, Jeff. *The Scrum Guide*. [S. l.]: Scrum.org; Scrum Inc., 2020. Disponível em: <https://scrumguides.org/scrum-guide.html>. Acesso em: 9 out. 2026.

SENAI. *Situação de Aprendizagem Integradora: da necessidade à solução digital: Desenvolvimento Mobile*. [S. l.]: SENAI, [s.d.]. Material didático.

SQLITE. *SQLite documentation*. [S. l.]: SQLite, [s.d.]. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: 9 out. 2026.

TESSERACT OCR. *Tesseract documentation*. [S. l.]: Tesseract OCR, [s.d.]. Disponível em: <https://tesseract-ocr.github.io/>. Acesso em: 9 out. 2026.